

SA-MIRI 2025

Practice Ph: Practical Guide to Efficient Training with PyTorch

Jakub Seliga (jakub.seliga@estudiantat.upc.edu)

Thomas Aubertier (thomas.aubertier@estudiantat.upc.edu)

Task 11.1 Find the Maximum Viable Batch Size

- Setting the dataset path

```
DS=/gpfs/projects/nct_345/datasets/micro-224
```

- Running from 16 to 512 (potentially)

```
BATCH_SIZES=(16 32 64 128 256 512)
```

- OOM :

```
>>>>>>>>>>>>>>>>>>>>>>>>>>>> Running with batch size 128  
Current hostname: as01r1b04  
[11/14/2025 10:13:18 AM] INFO - ##### Config #####  
[11/14/2025 10:13:18 AM] INFO - ### mixed_precision: None  
[11/14/2025 10:13:18 AM] INFO - ### batch_size: 128  
[11/14/2025 10:13:18 AM] INFO - ### eval_batch_size: 128  
[11/14/2025 10:13:18 AM] INFO - ### num_workers: 0  
[11/14/2025 10:13:18 AM] INFO - ### num_epochs: 5  
[11/14/2025 10:13:18 AM] INFO - ### epochs_eval: 5  
[11/14/2025 10:13:18 AM] INFO - ### iteration_logging: 500  
[11/14/2025 10:13:18 AM] INFO - ### model_name: vit  
[11/14/2025 10:13:18 AM] INFO - ### intermediate_dimensions: None  
[11/14/2025 10:13:18 AM] INFO - ### pretrained: False  
[11/14/2025 10:13:18 AM] INFO - ### compile: False  
[11/14/2025 10:13:18 AM] INFO - ### optimizer: sgd  
[11/14/2025 10:13:18 AM] INFO - ### learning_rate: 0.0005  
[11/14/2025 10:13:18 AM] INFO - ### dataset: /gpfs/projects/nct_345/datasets/micro-224  
[11/14/2025 10:13:18 AM] INFO - ### resolution: 224  
[11/14/2025 10:13:18 AM] INFO - ### Device: cuda  
[11/14/2025 10:13:18 AM] INFO - ### AMP Enabled: False  
[11/14/2025 10:13:18 AM] INFO - ### Total number of parameters 631021000  
[11/14/2025 10:13:18 AM] INFO - ### Total training samples (batches): 12416 (97)  
[11/14/2025 10:13:18 AM] INFO - #####  
[11/14/2025 10:13:18 AM] INFO - [EPOCH: 1] Starting training loop...<br></pre>
```

```

torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 482.00 MiB. GPU 0 has a total capacity of 63.43 GiB of which 117.50 MiB is free. Including non-PyTorch memory, this process has 63.30 GiB memory in use. Of the allocated memory 62.35 GiB is allocated by PyTorch, and 298.30 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management (https://pytorch.org/docs/stable/notes/cuda.html#environment-variables)

```

Task 11.1 Find the Maximum Viable Batch Size

Results :

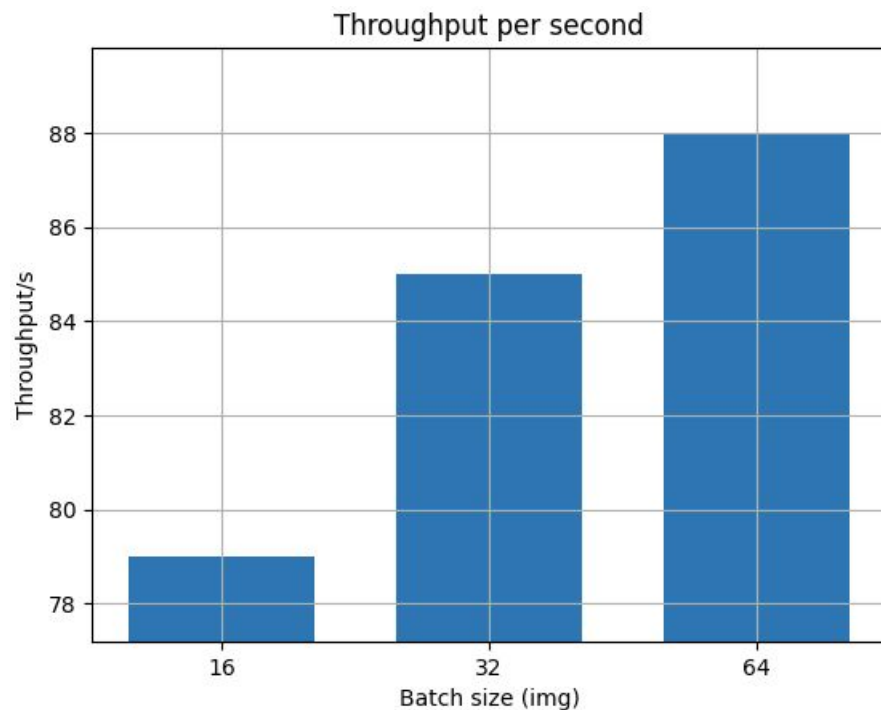
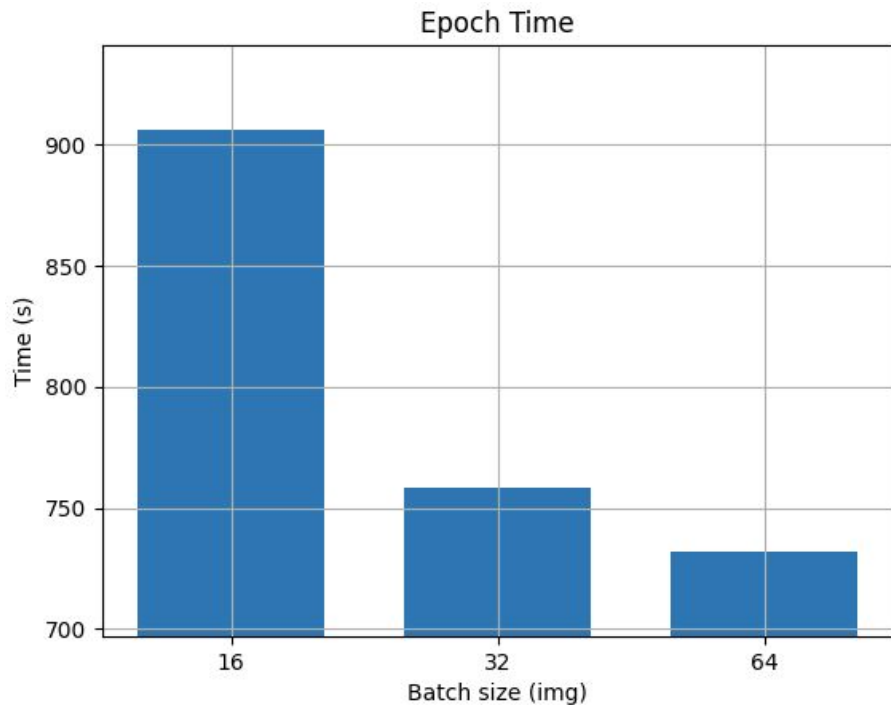
- Results similar to Table 11.1.
- Process finished up to batch size 64. Above = OOM
- Training time did slightly decreased / throughput increased, but memory usage is nearly proportional to batch size => not efficient.
- => Raising batch size is not necessary worth the cost, and can be unpractical due to memory quite fast.

Batch Size (image)	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
16	906	79	14
32	758	85	24
64	732	88	46
128	NA	NA	OOM

```
INFO - Training finished!  
INFO - Complete training Time: 906 s  
INFO - Training throughput: 79 imgs/s  
INFO - GPU memory reserved: 14 GB
```

Task 11.1 Find the Maximum Viable Batch Size

Results (barplots):



Task 11.2 Find the Maximum Viable Batch Size

Results (micro-224):

- Complete time is significantly lower than in Table 1.
- No process finished up with OOM
- Training time did steeply decreased up to 64, then stagnated.
- Memory usage is stable.
- => **custom** model scaling does reach its limit pretty fast (64), but is the fastest by miles.

Batch Size (image)	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
16	86	1108	3
32	45	1564	3
64	38	1832	3
128	36	1952	3
256	39	1781	3
512	36	1925	3

Task 11.2 Investigating the DataLoader Bottleneck with a Lightweight Model

- Note : Figure 11.2 was clearly obtained with `tiny-224`, so both datasets were tested with this model.

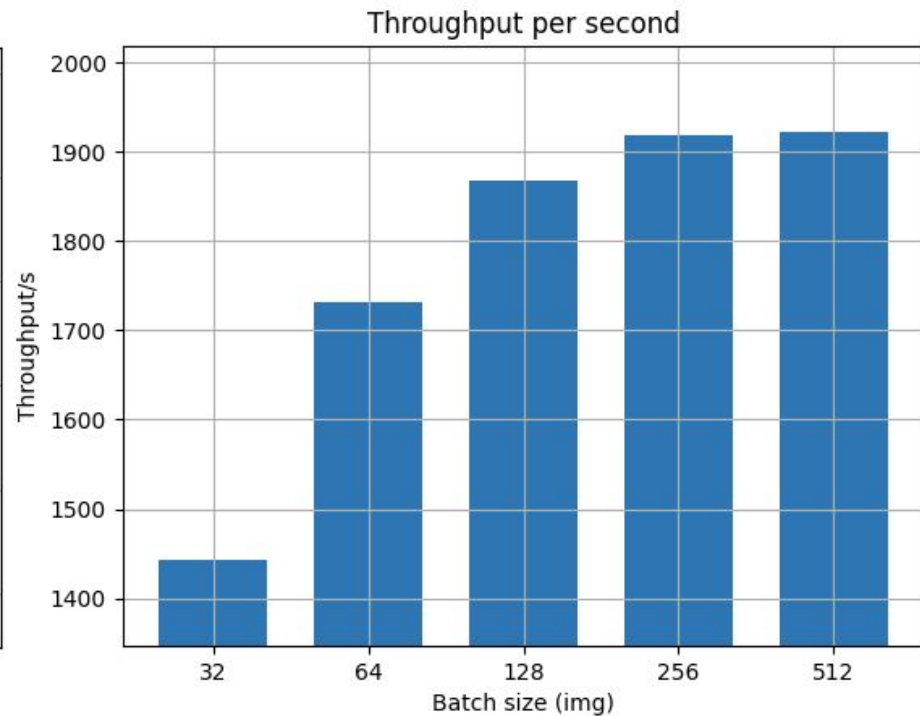
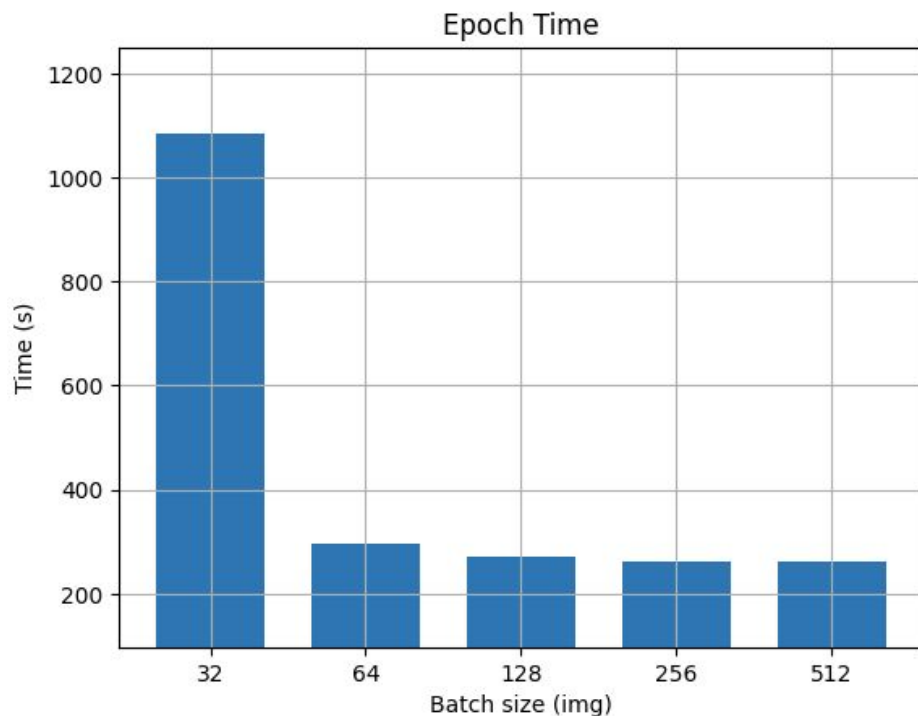
Results (`tiny-224`):

- Training time improved up to 64, throughput up to 256.
- No process finished up with OOM.
- Memory usage is stable.
- => `custom` model scaling does reach its limit pretty fast (64), regardless of `micro-224` or `tiny-224`.

Batch Size (image)	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
32	1085	1443	3
64	297	1732	3
128	272	1868	3
256	263	1919	3
512	262	1922	3

Task 11.2 Investigating the DataLoader Bottleneck with a Lightweight Model

Results (barplots for tiny-224):



Task 11.3 Optimizing DataLoaders with appropriate num_workers

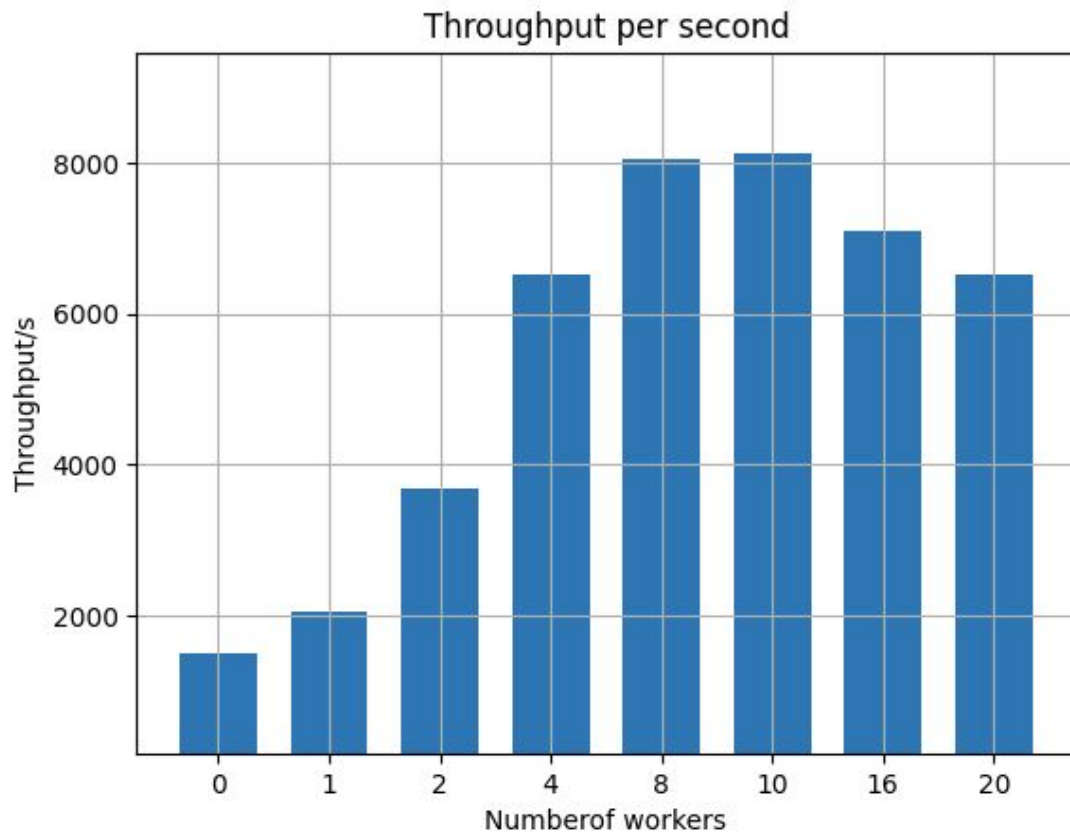
Results :

- Results similar to Table 11.3 for throughput, though training time scaled better.
- Time + throughput stopped improving at around 8/10 workers (even regressing at the end).
- Memory is not affected (constant).
- => Optimal number of workers would be 10

Number of workers	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
0	1212	1509	3
1	252	2054	3
2	56	3684	3
4	23	6507	3
8	14	8039	3
10	12	8123	3
16	12	7084	3
20	12	6511	3

Task 11.3 Optimizing DataLoaders with appropriate num_workers

Results (barplot):



Task 11.4 Confirming DataLoader Efficiency with ViT

Results :

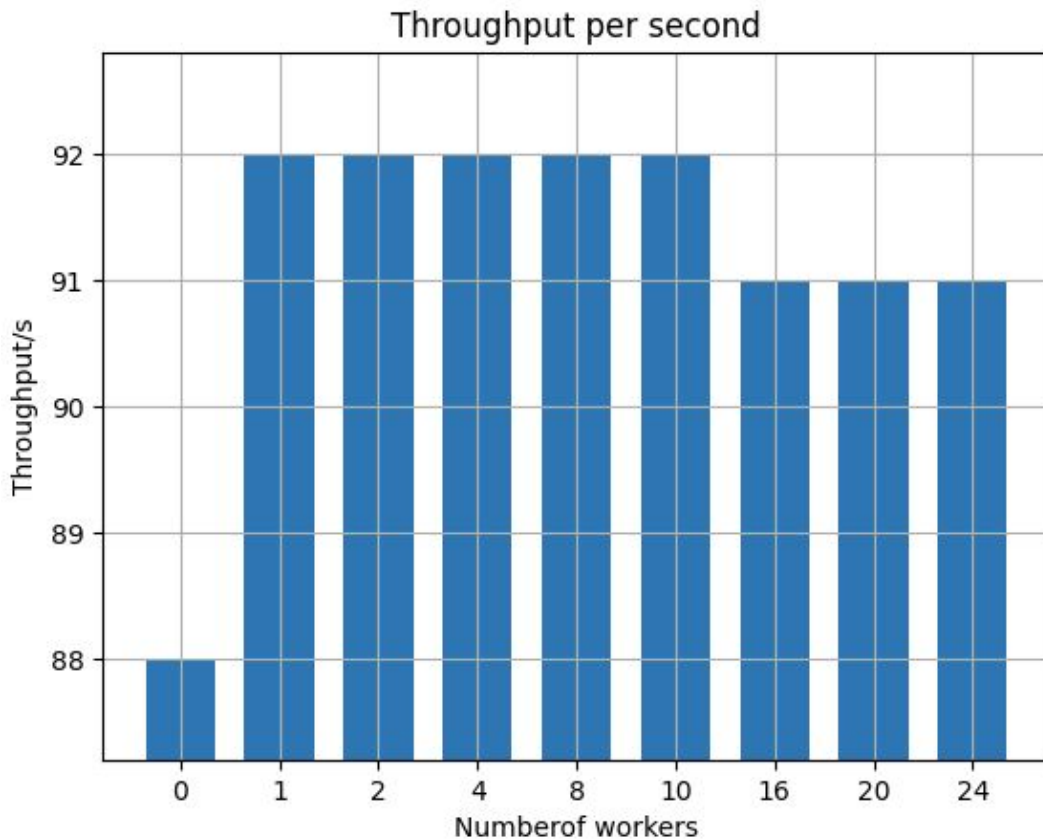
- Results very similar to Table 11.4 for training time and throughput,
- No scaling when using workers, only better than with 0.
- => No improvement but no additional cost either regarding those 3 metrics.

Number of workers	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
0	765	88	46
1	700	92	46
2	701	92	46
4	700	92	46
8	701	92	46
10	701	92	46
16	702	91	46
20	703	91	46
24	702	91	46

Task 11.4 Confirming DataLoader Efficiency with ViT

Results (barplot):

- Note : Do not be fooled !
Those variations are exaggerated by the plot y axis.



Task 11.5 Enable Mixed Precision with ViT + micro-224

- Processes were launched with `num_workers=10`.
- Using `--mixed_precision bf16` and `--mixed_precision fp16`.

```
PYTHON_FILE=./train.py
PYTHON_ARGS="--model_name $MODEL \
--dataset $DS \
--num_epochs $EPOCHS \
--batch_size $BS \
--eval_batch_size $BS \
--num_workers $NW \
--optimizer $OPTIM \
--iteration_logging $LOG_ITER \
--epochs_eval $EPOCHS_EVAL_FREQ \
--mixed_precision bf16 \
```

Task 11.5 Enable Mixed Precision with ViT + micro-224

- **fp32** nearly doubles the memory usage compared to **fp16** or **bf16** (expected) => Batch size 128 goes OOM.
- **fp16** and **bf16** are also 2x better in time and throughput, but does not scale between 64 and 128 batch size. Both have similar statistics, **bf16** being slightly better.

Precision	Batch Size (image)	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
fp32	64	703	92	46
fp32	128	-	-	OOM
fp16	64	362	182	27
fp16	128	342	190	50
bf16	64	353	186	27
bf16	128	334	196	50

Task 11.6 Reproducing the Effect of torch.compile()

JIT	Precision	Batch Size (image)	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
No	bf16	128	334	196	50
Yes	bf16	128	316	266	50

- Use of PyTorch's new compiler reduced training time and increased throughput
- Values are close to ones reported in book in Table 11.6.

Task 11.7 Report your Conclusions

- The most significant improvement is the model choice, which divided the overall training time by more than 10, with a stable memory usage. It's mainly due to its lower size.
- Data loaders were very efficient with the `custom` model (up to x100 faster, better parallelization) but did not affected `vit` in the slightest. That means `custom` was leaving GPUs idle quite a lot.
- Changing the float precision halved the training time at most for `vit` (great but not game changer), due to the size of variable.
- Just-In-Time compilation was the less significant change as the performances only affected the throughput by x1.5. The model was pretty well optimized in the first place, PyTorch did not altered its structure significantly.

Task 11.7 Report your Conclusions

Model	Precision	Number of workers	Batch Size (image)	JIT	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
Custom	default (fp32)	0	16	No	86	1108	3
	default (fp32)	0	256	No	39	1781	3
	default (fp32)	20	256	No	12	6511	3
Vit	default (fp32)	0	16	No	906	79	14
	default (fp32)	10	128	No	701	92	27
	bf16	10	128	No	334	196	50
	bf16	10	128	Yes	316	266	50

Task 11.7 Report your Conclusions

